

Heart Failure Prediction API Deployment Report

*Exploratory Analysis, Model Comparison, and Cloud
Deployment with a Secure REST API*

Arad Fadaei

Table of Contents

1. Executive Summary	1
2. Machine Learning Problem and Dataset	2
2.1. Machine Learning Problem	2
2.2. Dataset Used	2
2.3. Feature Set	2
3. Model Development Process	3
3.1. Exploratory Data Analysis (EDA)	3
3.2. Preprocessing	4
3.3. Training and Evaluation	4
4. Deployment Architecture Diagram and Cloud Services Used	6
4.1. Deployment Architecture Diagram	6
4.2. Cloud Services Used	6
4.3. Containerization and API	6
5. Challenges Faced and How They Were Overcome	7
5.1. Challenge 1: train/serve schema mismatch	7
5.2. Challenge 2: unseen categorical values at inference	7
5.3. Challenge 3: secure API key handling	7
5.4. Challenge 4: container architecture compatibility	7
6. Conclusion and Future Improvements	8
7. Screenshot Evidence of Working Deployment	9

Chapter 1. Executive Summary

This project implements a complete machine learning deployment workflow for heart disease risk prediction using the Kaggle Heart Failure Prediction dataset. The final output is a containerized FastAPI service deployed on Google Cloud Run, with API key authentication and real-time inference.

The system follows a train-once, infer-many design: model training occurs offline, while the deployed service only loads saved artifacts and performs prediction. This satisfies the assignment constraint against training during inference requests.

Two models were trained and compared:

- Logistic Regression
- Random Forest Classifier

Random Forest was selected based on F1 score and overall balanced performance.

 heart-failure-api-00004-mc8

Deployed by aradfadaie@gmail.com using Cloud Console

Containers

Networking

Security

YAML

General

Billing	Request-based
Startup CPU boost	Enabled
Concurrency	80
Request timeout	300 seconds
Execution environment	Default

Auto-scaling

Revision max. instances	3
-------------------------	---

Image	mirror.gcr.io/downmoto/heart-failure-api@sha...
Port	8080
Build	(no build information available) ?
Source	(no source information available) ?
Command and arguments	(container entrypoint)
CPU limit	1
Memory limit	512MiB

Chapter 2. Machine Learning Problem and Dataset

2.1. Machine Learning Problem

The task is binary classification: predict whether a patient is likely to have heart disease from structured clinical and demographic features.

Target variable:

- `HeartDisease` where `0` means no heart disease predicted and `1` means heart disease predicted.

2.2. Dataset Used

- Dataset source: Kaggle (`fedesoriano/heart-failure-prediction`)
- Local file used for training: `data/heart.csv`
- Data loading helper: `scripts/download_dataset.py`

2.3. Feature Set

The model uses these inputs:

- age
- sex
- chest pain type
- resting blood pressure
- cholesterol
- fasting blood sugar flag
- resting ECG
- max heart rate
- exercise angina
- oldpeak
- ST slope

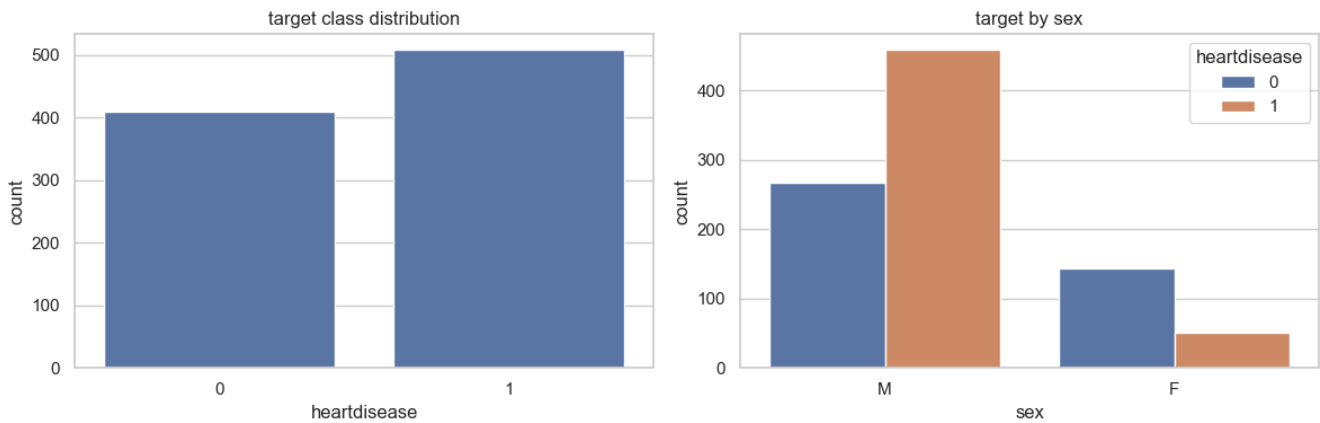
Chapter 3. Model Development Process

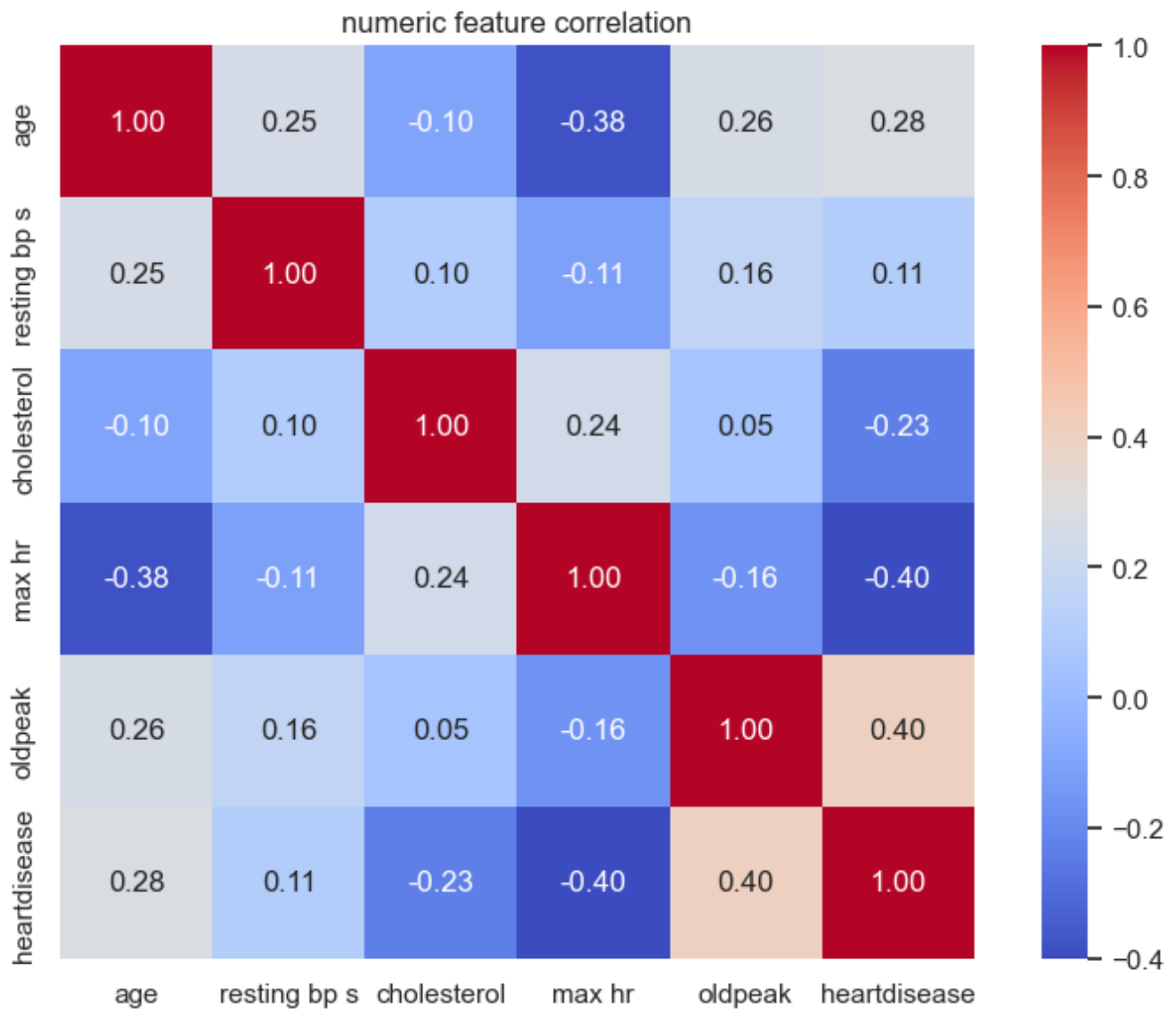
3.1. Exploratory Data Analysis (EDA)

EDA was performed in `EDA.ipynb` to understand distribution, class balance, and relationships between variables.

EDA outputs include:

- target class distribution
- target split by sex
- numeric feature distributions by target
- boxplots by target
- correlation heatmap
- grouped risk rates by key categories





3.2. Preprocessing

Preprocessing is implemented with `ColumnTransformer` in `training/train.py`:

- numeric features: median imputation + standard scaling
- categorical features: mode imputation + one-hot encoding (`handle_unknown="ignore"`)

This ensures the same transformations are used during training and inference, reducing train/serve skew.

3.3. Training and Evaluation

Two models were trained on the same stratified split (`test_size=0.2`, `random_state=42`):

- Logistic Regression (`class_weight="balanced"`)
- Random Forest (`n_estimators=400`)

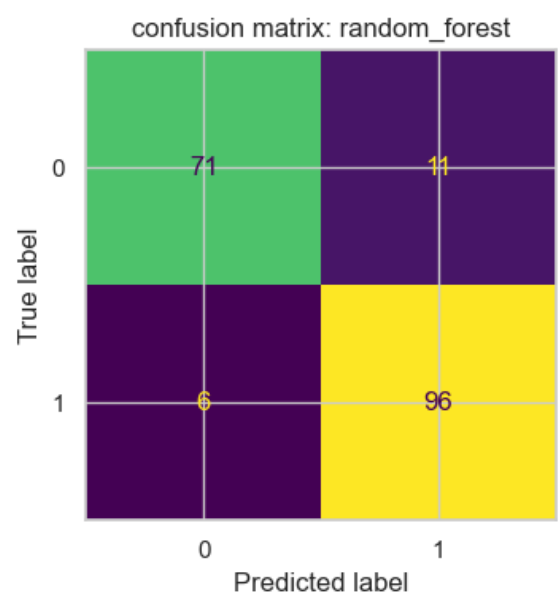
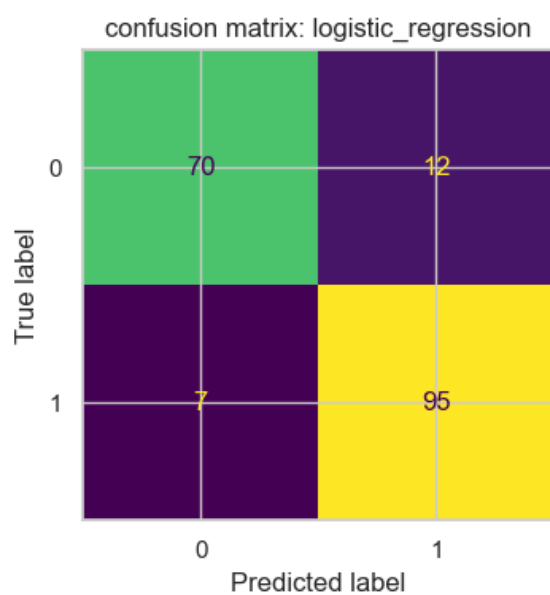
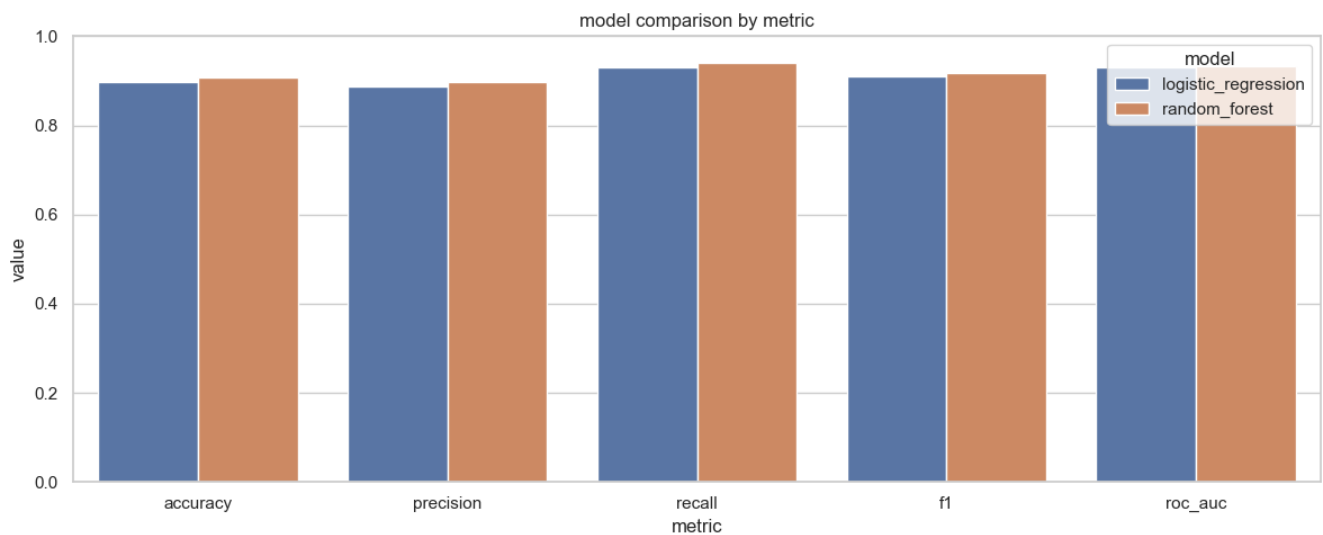
Evaluation metrics used:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

Evaluation summary:

Model	Accuracy	Precision	Recall	F1	ROC AUC
Logistic Regression	0.8967	0.8879	0.9314	0.9091	0.9296
Random Forest	0.9076	0.8972	0.9412	0.9187	0.9316

Selected model: Random Forest (highest F1).

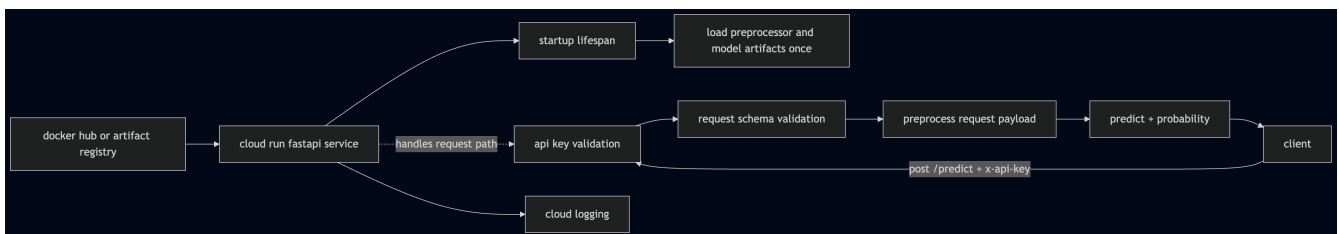


Chapter 4. Deployment Architecture Diagram and Cloud Services Used

4.1. Deployment Architecture Diagram

Architecture summary:

1. Client sends **POST /predict** with JSON payload and **x-api-key**
2. Cloud Run hosts the FastAPI container
3. API key is validated
4. Preprocessor and model artifacts are loaded and applied
5. Prediction and probability are returned
6. Logs are captured in Cloud Logging



4.2. Cloud Services Used

- Google Cloud Run: serverless container hosting for API
- Docker Hub image source: container image storage
- Cloud Logging: request and runtime logs

4.3. Containerization and API

Containerization is implemented with Docker:

- Base image: **python:3.12-slim**
- App server: **uvicorn src.main:app --host 0.0.0.0 --port 8080**
- Runtime secret: **API_KEY** environment variable
- Endpoint behavior:
 - **GET /health** for service checks
 - **POST /predict** for authenticated inference

Chapter 5. Challenges Faced and How They Were Overcome

5.1. Challenge 1: train/serve schema mismatch

I ran into a schema mismatch early when I moved from notebook exploration to API serving. The Kaggle dataset column names (for example `ChestPainType`, `RestingBP`) did not match the JSON field naming I wanted in the API payload.

How I solved it: I standardized column names in the training pipeline and used explicit Pydantic aliases in the API schema. This made the data contract deterministic between training and inference.

5.2. Challenge 2: unseen categorical values at inference

I also had to handle the case where inference-time categorical values might differ from what the model saw during training.

How I solved it: I configured `OneHotEncoder(handle_unknown="ignore")` in the preprocessing pipeline, which prevented runtime crashes from previously unseen categories.

5.3. Challenge 3: secure API key handling

During deployment hardening, I needed to secure the endpoint without exposing secrets in source code or container layers.

How I solved it: I moved key handling to runtime environment variables (`API_KEY`) and kept only `.env.example` as a template. This kept secrets out of the codebase while preserving reproducibility.

5.4. Challenge 4: container architecture compatibility

This was the most concrete deployment blocker I faced: my first Cloud Run deployment failed because the image built on Apple Silicon did not provide a compatible `linux/amd64` manifest.

How I solved it: I rebuilt and pushed the container with `docker buildx --platform linux/amd64`, then redeployed successfully.

Chapter 6. Conclusion and Future Improvements

This project successfully delivers a complete ML deployment pipeline, from EDA and model comparison to a live cloud-hosted API with authentication. The final implementation meets the assignment requirements and demonstrates an end-to-end operationalized workflow.

Potential future improvements:

- integrate Secret Manager for stronger secret governance
- add CI/CD for automated build, test, and deployment
- implement monitoring for drift and performance changes
- introduce model versioning and rollback strategy

Chapter 7. Screenshot Evidence of Working Deployment

1. Successful **POST** `/predict` response from deployed endpoint
2. Unauthorized **POST** `/predict` response (**401**) from deployed endpoint
3. Cloud logs showing request handling

This screenshot shows a successful **POST** request to `https://heart-failure-api-1053145910416.europe-west1.run.app/predict`. The status is **200 OK**, the size is **43 Bytes**, and the time is **254 ms**. The response body is a JSON object:

```
{  "predictions": [    0  ],  "probabilities": [    0.005  ]}
```

This screenshot shows an unauthorized **POST** request to `https://heart-failure-api-1053145910416.europe-west1.run.app/predict`. The status is **401 Unauthorized**, the size is **28 Bytes**, and the time is **200 ms**. The response body is a JSON object:

```
{  "detail": "invalid api key"}
```

This screenshot shows a list of logs in Google Cloud. The logs are filtered by severity (Default) and show a series of **POST** requests to `/predict` with a status of **200 OK**. The logs are sorted by time, and the first log entry is expanded to show the following details:

```
{  insertId: "6998ac2c000247c446d3727"  labels: {1}  logName: "projects/project-651d62bc-8fda-4f61-84b/logs/run.googleapis.com%2Fstdout"  receiveTimestamp: "2026-02-20T18:47:08.208241898Z"  resource: {2}  textPayload: "INFO: 169.254.169.126:35918 - \"POST /predict HTTP/1.1\" 200 OK"  timestamp: "2026-02-20T18:47:08.009340Z"}
```